

实验报告：基于 AI 协同开发的任务管理系统设计与实现

一、实验内容、要求或目的

1. 实验目的：

- **语言基础与编码能力：**熟悉 Go 语言在小型控制台系统开发中的应用，掌握核心的语言特性，包括结构体定义（Struct）、切片操作（Slice）、包组织（Packages）等基础程序设计能力。
- **数据持久化机制：**深入理解数据模型与持久化原理，掌握利用 Go 标准库 os 和 encoding/json 实现对象数据的序列化内存写入与硬盘文件的映射保存。
- **工程化与 AI 赋能：**学习并实践利用大语言模型开发工具辅助开发，覆盖从前期的需求分析、系统构建策略到后期的代码排错与重构。
- **逻辑结构延展：**在一套标准 CRUD（增删改查）生命周期基础上，结合防篡改记录的密码学“哈希链”思想，完成系统结构与安全模块的综合设计。

2. 实验任务与内容：

本次实验采用 Go 语言设计并实现了一套基于命令行的“个人健身训练任务管理系统”。系统重点围绕任务实体展开数据的生命周期管理，实现了如下功能：

- **基础功能：**独立设计包含任务 ID、名称、描述、状态、创建时间及优先级的完整数据结构。实现健身任务的创建、结构化查询、状态修改以及任务删除。
- **拓展功能：**新增了任务分类（如：有氧/无氧）和截止时间。
- **加分项实践（防篡改操作追踪）：**独立实现了一套基于链式日志的记录系统。每一次影响持久化数据的操作增删改均会生成一条摘要，附加时间戳并结合上一条记录的哈希针通过 SHA-256 算法加密生成块数字指纹。最后提供日志独立校验引擎，用于鉴定底层 json 文本数据是否曾受到外部非法篡改。

二、AI 参与过程

由于课堂理论知识吸收存在断层，特别在 Go 语言特有的数据结构（如切片与指针概念）及持久化操作上不够熟练。本次实验全流程借用了 AI 辅助，主要互动过程复盘如下：

1. 如何描述需求与基础框架的搭建：

- **我的描述：**我对 Go 语言的面向对象概念不太懂。我需要写一个命令行版本的健身任务管理系统，要存入 JSON 文件。此外我要做一个像区块链一样的操作记录并追踪记下每步操作的哈希作为加分项。请告诉我代码应该怎么组织，以及怎样定义这些数据结构。
- **AI 的帮助：**AI 并没有直接扔给我一堆杂乱的代码，而是先向我解释了 Go 语言中没有 Class，而是通过 struct 来聚合数据。AI 给出了 FitnessTask 和 LogBlock 的结构原型，并重点提醒我：**要让 encoding/json 包成功导出字段，结构体名的首字母以及字段的首字母都必须大写**；同时可以利用反引号（如 json:"id"）来映射 JSON 文件里的键名。这个特性解答了我之前自学时的很大困惑。

2. 核心算法探究与疑难 Bug 调试：

- **遇到的问题：**在实现删除功能时，我最初尝试将数组中对应该任务的索引置空，但总是导致访问越界或后续打印空指针错误。
- **AI 的指导：**AI 教会了我 Go 独有的切片拼接语法 `tasks = append(tasks[:i], tasks[i+1:]...)`。AI 非常生动地帮我剖析了内存模型：切片在 Go 中是指向底层连续数组的窗口，上述语法的作用就是截走前半段和后半段，然后利用 ... 操作符将后半段元素打散重组到前半段，实现了优雅高效的删除。
- **加密链算法调整：**在处理链式日志加分项时，AI 帮我理清了 crypto/sha256 包的调用流程。最初用 `time.Now().String()` 拼接内容造成每次重新校验都哈希不匹配，AI 指导我改用 `time.Format(time.RFC3339)` 输出了固定格式，完美攻克了哈希碰撞断裂的 Bug。

3. 对 AI 生成结果的修改或调整：

- 初期 AI 提供的交互代码存在大量冗余的打印，比如每个回显都会生硬地拼接缺乏工程感的颜文字。为了充分内化代码，我让命令行界面完全回归极其专业的工程师后台。
 - 在文件读写模块，AI 生成的代码对文件不存在的边界情况处理较为粗暴。我引入了 `os.IsNotExist` 等规范化错误捕捉，使得系统在初次启动也就是无档案时能平滑生成基础数据表，而不是直接抛出错误。
 - 针对每一行核心逻辑，我不只限于表层的增删改查。对所有被 AI 包装的高级用法进行逐行剖析，自己重构了业务代码，并补全了基于工程逻辑的中文注释。
-

三、程序主要算法或源代码

按照课程要求：因不再提交源代码文件，将部分核心结构与算法代码展示如下
完整源代码在最后

1. 利用 Struct Tag 实现数据模型定义与 JSON 化

Go 通过结构体聚合属性，并巧妙使用 Tag 技术实现序列化：

// 健身任务核心数据模型

```
type FitnessTask struct {  
    ID string json:"id"  
    Name string json:"name"  
    Description string json:"description"  
    Category string json:"category" // 扩展项：健身分类  
    Status string json:"status" // 状态：待处理/进行中  
    Priority int json:"priority" // 优先层级  
    CreatedAt time.Time json:"created_at"  
    Deadline time.Time json:"deadline" // 扩展项：截止时间  
}
```

2. 基于切片的 CRUD 核心逻辑（以删除操作为例）

切片在 Go 中代表动态数组窗，在处理元素的物理移除时，利用了切片前后闭包提取结合 ... 解包动作实现内存平移覆盖。

```
// 从切片中移除对应的任务 ID 索引 i  
tasks = append(tasks[:i], tasks[i+1:]...)  
// 在操作系统修改记录写入防篡改链  
recordLog(fmt.Sprintf("删除任务 [%s]", id))
```

3. 链式防篡改密码学记录引擎（加分项）

系统利用前置哈希指针，将每次发生的数据变动串联。一旦底层 JSON 被直接利用记事本强行修改，程序下一次开启便能自动拦截。

```
// Hash 发生器：接收时间、动作并包含前置块 Hash 组合文本基料加密
```

```

func calculateHash(index int, timestamp time.Time, summary, prevHash string)
string {
    record := strconv.Itoa(index) + timestamp.Format(time.RFC3339) + summary +
prevHash

    h := sha256.New()

    h.Write([]byte(record)) // 必须将 string 换算至 []byte

    return hex.EncodeToString(h.Sum(nil))
}

// 防篡改校验核心循环器
func verifyChain() bool {
    for i := 1; i < len(blockchain); i++ {
        prev := blockchain[i-1]
        curr := blockchain[i]

        // 1. 本块存放的 PrevHash 是否与上一实体 Hash 一致 (防抽轨)
        if curr.PrevHash != prev.Hash { return false }

        // 2. 本块现存正文重新结算是否匹配现存 Hash (防篡改内容)
        if curr.Hash != calculateHash(curr.Index, curr.Timestamp, curr.Summary,
curr.PrevHash) {
            return false
        }
    }

    return true
}

```

四、实验结果与分析

运行环境：终端命令行 (powershell / bash) + go run main.go。

系统功能与预期验证：

1. **基础 CRUD：**启动时系统通过读取不到配置判断，顺利初始化了创世区块日志。调用增改删测试：控制台展现标准的制表符对齐列表，状态流转完全符合业务预期，功能完善。

2. **数据持久化验证：**主动中断程序退出进程。查看工作区目录，发现已平稳落盘产生 tasks_data.json 与 chain_logs.json。重新挂载运行程序，旧数据 100%全景还原，JSON 编码及解码功能达到设计预期。
3. **加分项：黑盒篡改拦截测试：**在未被篡改时，系统返回：验证结果：日志链完整，均未被外部篡改！。随后，我强行用文本编辑器打开 chain_logs.json 修改了内部一条操作描述的字眼，系统立即提示：防篡改链条断裂或指纹不匹配。加密算法和指纹校验逻辑完全发挥了防御效应。

测试图例如下

Task_data.json

```
fitness_task_manager > {} tasks_data.json > ...
1  [
2    {
3      "id": "1778155636",
4      "name": "深蹲训练",
5      "description": "完成四组深蹲, 每组15次, 休息1分钟",
6      "category": "无氧锻炼",
7      "status": "待处理",
8      "priority": 4,
9      "created_at": "2026-05-07T20:07:16.4801372+08:00",
10     "deadline": "2026-05-08T20:07:16.4801372+08:00"
11   },
12   {
13     "id": "1778155675",
14     "name": "五公里晨跑",
15     "description": "保持在6分半配速内跑完5km",
16     "category": "有氧锻炼",
17     "status": "待处理",
18     "priority": 5,
19     "created_at": "2026-05-07T20:07:55.9359723+08:00",
20     "deadline": "2026-05-08T20:07:55.9359723+08:00"
21   }
22 ]
```

Chain_log.json

```

fitness_task_manager > {} chain_logs.json > ...
1  [
2    {
3      "index": 0,
4      "timestamp": "2026-05-07T20:06:18.9833836+08:00",
5      "summary": "系统初始化 - 创建创世操作记录",
6      "prev_hash": "0000000000000000000000000000000000000000000000000000000000000000",
7      "hash": "ea83adce1749b4d2fcc98900bd6fcc38fe614304e12d20da85fe65aef35acdb55"
8    },
9    {
10     "index": 1,
11     "timestamp": "2026-05-07T20:07:16.4801372+08:00",
12     "summary": "新增任务: [1778155636] 深蹲训练",
13     "prev_hash": "ea83adce1749b4d2fcc98900bd6fcc38fe614304e12d20da85fe65aef35acdb55",
14     "hash": "4157b8d8f81aa063b4871537da0fee9fd87060b3a2f51743a4e6f3b296a8b9d7"
15   },
16   {
17     "index": 2,
18     "timestamp": "2026-05-07T20:07:55.937992+08:00",
19     "summary": "新增任务: [1778155675] 五公里晨跑",
20     "prev_hash": "4157b8d8f81aa063b4871537da0fee9fd87060b3a2f51743a4e6f3b296a8b9d7",
21     "hash": "9fcb8ff4ef362061cfcdb9c0d4669de58f53a19595756f01640ce27730a03e1a"
22   }
23 ]

```

篡改结果

```

===== 任务管理系统 =====
1. 新增任务
2. 查看任务
3. 更新状态
4. 删除任务
5. 校验底层操作记录防篡改链
0. 退出系统

=====
请选择操作编号: 5
验证结果: 日志链完整, 所有历史数据均未被外部篡改!

```

```

===== 任务管理系统 =====
1. 新增任务
2. 查看任务
3. 更新状态
4. 删除任务
5. 校验底层操作记录防篡改链
0. 退出系统

=====
请选择操作编号: 5
警告检测: 发现链条断裂或指纹不匹配, 记录文件可能已被文本编辑器强行修改过!

```


五、实验结论与心得体会

1. 编程技术收获:

我跨越了书本概念，切实领悟到了 Go 语言的系统级特性，它剔除了传统语言继承与封装，靠着精简的 `Struct` 配合 `Slice` 切片展开机制就能高效管理和操作大型数据。此外标准库包含序列化与系统加密功能可谓开箱即用。

2. 关于利用 AI 辅助开发的深刻体会:

最真实的体会是：AI 应当被用作学习的工具，而非死板的代写脚本。如果全盘照抄，最终生成的程序会塞满机器味极重的回显话术。虽然 ai 生成确实方便，但是依然要知其所以然，学习的过程不能以结果性为导向，而是需要掌握这项语言，不拘泥于普通的增删改查，把精力放在 ai 无法完全控制的上面，当然简单的有些重要的代码依旧需要人工审核。所以我的首要操作是扒掉它的皮，针对底盘像 `crypto/sha256` 为什么在计算时必须传递字节序列 `[]byte` 这类疑问，主动拷问原理，并最后注释留存进自己的代码。看透了底层运转的脉络，亲自重构逻辑，这才是学习工程师素养的最佳路径。

六、课后习题练习情况记录（随堂练习存档）

根据实验课安排，在此记录课后核心代码练习的完成情况。

第一次习题练习（第四章：控制结构）

1. 第 16 题：水仙花数

- **题目说明：**编程输出所有的水仙花数（三位的十进制整数，其各个数位上数字的立方和恰好等于这个整数本身）。
- **解题思路：**使用 `for` 循环遍历 100 至 999，在循环内分别使用整除（/）和取余（%）操作提出百位、十位和个位，运算立方和并与原数字进行对比验证输出。
- **核心代码：**

```
package main

import "fmt"

func main() {

    fmt.Print("所有的水仙花数有: ")

    for i := 100; i < 1000; i++ {

        a := i / 100 // 分离百位
```

```

    b := (i / 10) % 10 // 分离十位
    c := i % 10 // 分离个位
    if aaa + bbb + ccc == i {
        fmt.Printf("%d ", i)
    }
}
fmt.Println()
}

```

2. 第 18 题：最大公约数与最小公倍数

- **题目说明：**输入两个正整数 m 和 n ，求其最大公约数 gcd 和最小公倍数 lcm 。
- **解题思路：**最快方案为应用辗转相除法配合 Go 多重赋值特性
($m, n = n, m\%n$) 回避老式的临时临时变量(temp 变量)操作。最小公倍数套用两数乘积除以最大公约数的公式求出。
- **核心代码：**

```

package main

import "fmt"

func gcd(m, n int) int {
    for n != 0 {
        m, n = n, m%n // Go 语言多重赋值迭代
    }
    return m
}

func lcm(m, n int) int {
    return (m * n) / gcd(m, n)
}

func main() {
    var m, n int
    fmt.Print("请输入两个正整数 m 和 n: ")
}

```



```
fmt.Scanf("%d %d", &m, &n)

fmt.Printf("最大公约数是: %d\n", gcd(m, n))

fmt.Printf("最小公倍数是: %d\n", lcm(m, n))
}
```

第二次习题练习（第五章：函数与方法）

3. 第 21 题：使用闭包创建偶数生成器

- **题目说明：**使用闭包创建一个偶数生成器 `evenGenerator()`。
- **解题思路：**在一个函数内定义闭包函数作为返回值。在外部方法内预置状态变量 `i := 0`，匿名内部闭包执行对该变量的读写修改运算并使得其生命期被持久锁定。
- **核心代码：**

```
package main

import "fmt"

func evenGenerator() func() int {
    i := 0 // 外部环境变量，由下方闭包捕获生命期
    return func() int {
        ret := i
        i += 2 // 每次调用自增保存 2
        return ret
    }
}

func main() {
    nextEven := evenGenerator() // 取得闭包生成器句柄
    fmt.Println(nextEven()) // 0
    fmt.Println(nextEven()) // 2
    fmt.Println(nextEven()) // 4
}
```

第三次习题练习（第六章：数据容器）

4. 第 13 题：切片追加与解包

- **题目说明：**编程实现：为切片 `s` 追加一个新切片 `[]int{1, 2}`。
- **解题思路：**切片追加利用内置函数 `append()`，但传入另一个切片时需要注意直接添加会导致类型冲突。使用 Go 内置的展开语法糖 `...` 将目标切片“解包”为单个可迭代参数再合并。
- **核心代码：**

```
package main

import "fmt"

func main() {
    s := []int{8, 9, 10}
    newSlice := []int{1, 2}
    // 核心步骤：使用 ... 将新数组解包成独立参数传入 append 中
    s = append(s, newSlice...)
    fmt.Printf("追加后的切片: %v\n", s)
}
```

5. 第 20 题：列表（List）的操作与遍历

- **题目说明：**在列表 `lt` 的头部增加元素 `5`，在其尾部依次增加 3 个元素 `"b"`、`"c"`、`"a"`。遍历该列表，输出其元素的值。
- **解题思路：**引入双向链表核心模块 `container/list` 构造列表，随后通过暴露的方法 `PushFront` 首部以及 `PushBack` 尾部追加，指针遍历采用前驱遍历器并结合泛型方法获取。
- **核心代码：**

```
package main

import (
    "container/list"
    "fmt"
)

func main() {
```

```

lt := list.New() // 实例化
lt.PushFront(5) // 头部追加
lt.PushBack("b") // 尾部推入
lt.PushBack("c")
lt.PushBack("a")
fmt.Print("链表遍历输出: ")
// 通过 Front 方法提取首结点做为其实游标，并沿 Next() 不断检索
for e := lt.Front(); e != nil; e = e.Next() {
    fmt.Printf("%v ", e.Value)
}
fmt.Println()
}

```

第四次习题练习（第八章：常用标准库）

第 20 题：字符串切片的升序排列

- **题目要求：** 已知字符串切片 `s := []string{"b", "a", "d", "aa", "c"}`，编写程序将其升序排列并输出。

- **算法讲解：**

在 Go 语言中，处理切片排序最标准且高效的方式是使用标准库中的 `sort` 包。对于字符串切片，可以直接调用 `sort.Strings()` 函数。该函数会直接修改原切片（就地排序），按照字符串的 **ASCII 字典序**（例如 "a" 会排在 "aa" 之前，"aa" 会排在 "b" 之前）进行升序排列。

- **代码实现：**

```

package main

import (
    "fmt"
    "sort"
)

func main() {

```

```
// 初始化切片
s := []string{"b", "a", "d", "aa", "c"}
fmt.Println("排序前:", s)
// 调用标准库函数进行升序排列
sort.Strings(s)
// 输出结果
fmt.Println("排序后:", s)
}
```

第五次习题练习（第九章：并发编程）

第 9 题：协程与通道的随机数通信

- **题目要求：** 启动一个协程生成 5 个 0 到 100 之间的随机整数，通过通道发送给主协程，主协程接收并输出。
- **算法讲解：**
 - 初始化与同步：** 使用 `make(chan int)` 创建一个整型通道，作为子协程与主协程之间的通信媒介。
 - 并发生产：** 利用 `go` 关键字启动一个匿名函数作为子协程。在其中循环 5 次，使用 `math/rand` 生成指定范围内的随机数。
 - 资源关闭：** 子协程完成发送后，必须显式调用 `close(ch)`。这是为了告诉主协程数据已发完，防止主协程在遍历通道时进入死锁。
 - 顺序消费：** 主协程使用 `for range` 结构阻塞式地等待通道数据，每接收到一个数字便立即打印。
- **代码实现：**

```
package main
import (
    "fmt"
    "math/rand"
    "time"
```

```

)
func main() {
    // 设置随机数种子，确保每次运行结果不同
    rand.Seed(time.Now().UnixNano())
    // 创建一个用于传递整数的通道
    ch := make(chan int)
    // 启动子协程（生产者）
    go func() {
        fmt.Println("[子协程] 开始生成随机数...")
        for i := 0; i < 5; i++ {
            num := rand.Intn(101) // 生成 [0, 100] 的随机数
            ch <- num // 将数据送入通道
        }
        close(ch) // 关闭通道，通知主协程发送完毕
    }()
    // 主协程（消费者）通过 range 持续接收数据
    fmt.Println("[主协程] 等待接收数据：")
    for val := range ch {
        fmt.Printf("接收到数字: %d\n", val)
    }
    fmt.Println("程序执行完毕。")
}

```

最后源码附上

```
package main

// 引入 Go 语言的标准库
import (
    "bufio"    // 用于处理带缓冲的 I/O 读取，比如读取用户的整行输入
    "crypto/sha256" // 密码学库，用于计算 SHA-256 哈希值（用于防篡改链）
    "encoding/hex" // 用于将哈希计算后的二进制结果转为十六进制字符串
    "encoding/json" // 用于将 Go 语言的结构体数据与 JSON 格式相互转换（持久化核心）
    "fmt"        // 格式化 I/O，用于在终端打印输出
    "os"         // 操作系统交互，用于文件读写操作
    "strconv"    // 字符串转换库，用于字符串与数字类型之间的转换
    "strings"    // 字符串处理库，用于去除输入两端的空格等
    "time"       // 时间处理库，用于记录任务创建和截止时间
)

// ===== 数据模型 =====

// FitnessTask 任务数据结构体
// 结构体(Struct)是 Go 语言中用于组合多个变量的自定义类型，类似于其他语言的类(Class)
// 后面的 `json:"..."` 是结构体标签(Tag)，告诉 json 库在保存文件时该字段叫什么名字
type FitnessTask struct {
    ID      string `json:"id"`    // 唯一标识符
    Name    string `json:"name"`  // 任务名称
    Description string `json:"description"` // 任务详细描述
}
```

```

    Category string `json:"category"` // 分类扩展字段
    Status   string `json:"status"`   // 当前状态：待处理、进行中、已完成
    Priority  int    `json:"priority"`  // 优先级整数：1-5
    CreatedAt time.Time `json:"created_at"` // 创建时的系统时间
    Deadline  time.Time `json:"deadline"` // 扩展字段：截止时间
}

// LogBlock 链式日志块结构体 (用于操作追踪与防篡改)
type LogBlock struct {
    Index    int    `json:"index"` // 区块所在索引(第几个操作)
    Timestamp time.Time `json:"timestamp"` // 发生此操作的时间
    Summary  string `json:"summary"` // 做了什么的文本摘要，例如"新增任务A"
    PrevHash string `json:"prev_hash"` // 上一次操作生成的哈希值(将所有操作串联起来的关键)
    Hash     string `json:"hash"`    // 本次操作生成的哈希值
}

// 全局变量声明
var {
    // 切片(Slice)：类似于动态数组，可以随时追加元素。由于还没有学习数据库，
    先用切片把数据全放在内存里。

    tasks []FitnessTask
    blockchain []LogBlock

    // 数据持久化保存的文件路径
    taskFile = "tasks_data.json"
    chainFile = "chain_logs.json"

```



```
)
```

```
// ===== 数据持久化与链式日志核心机制 =====
```

```
// loadData 程序启动时从文件读取 JSON 数据到内存
```

```
func loadData() {
```

```
    // os.ReadFile 会把整个文件读成字节数组 ([]byte)
```

```
    if file, err := os.ReadFile(taskFile); err == nil {
```

```
        // 返回没有错误(err == nil)说明文件存在，通过 Unmarshal 把字节转回 tasks  
        切片
```

```
        json.Unmarshal(file, &tasks)
```

```
    }
```

```
    // 读取日志链文件
```

```
    if cFile, err := os.ReadFile(chainFile); err == nil {
```

```
        json.Unmarshal(cFile, &blockchain)
```

```
    } else {
```

```
        // 如果读取失败(通常是因为第一次运行还没有文件)，则记录一条创世日志启  
        动系统
```

```
        recordLog("系统初始化 - 创建创世操作记录")
```

```
    }
```

```
}
```

```
// saveData 将内存里的数据结构重新持久化保存为硬盘上的文件
```

```
func saveData() {
```

```
    // MarshalIndent 将结构体变成带缩进、易于人类阅读的 JSON 格式
```

```
    data, _ := json.MarshalIndent(tasks, "", " ")
```

```
    // os.WriteFile 写入文件，0644 表示文件权限(可读写)
```

[illegible]

```

    // 取出 blockchain 数组最后一个元素的 Hash 作为当前元素的 PrevHash
    prevHash = blockchain[index-1].Hash
}

timestamp := time.Now()
// 计算本次操作的指纹
hash := calculateHash(index, timestamp, summary, prevHash)

// 把新的操作日志追加进去
blockchain = append(blockchain, LogBlock{
    Index:  index,
    Timestamp: timestamp,
    Summary: summary,
    PrevHash: prevHash,
    Hash:    hash,
})

// 每次有了新操作后，立即保存数据文件
saveData()
}

// verifyChain 遍历检查日志记录有没有被人在外部偷偷修改文本
func verifyChain() bool {
    // 从第 1 个(由 0 开始)遍历到最后一个
    for i := 1; i < len(blockchain); i++ {
        prev := blockchain[i-1]
        curr := blockchain[i]

```

```

    // 1. 检查链条是否断裂：当前块记录的"上一步哈希" 是否等于 上一块真实的
    哈希
    if curr.PrevHash != prev.Hash {
        return false
    }

    // 2. 检查内容是否被篡改：用现有内容重新计算哈希，看看和原来存下来的哈
    希能否对上
    if curr.Hash != calculateHash(curr.Index, curr.Timestamp, curr.Summary,
    curr.PrevHash) {
        return false
    }
}
return true
}

// ===== 业务逻辑处理 =====

// readInput 封装一个通用的命令行输入读取函数，带回显提示
func readInput(prompt string) string {
    fmt.Print(prompt)
    reader := bufio.NewReader(os.Stdin) // 绑定标准输入终端
    // 读一直到用户按下回车键 ('\n')
    input, _ := reader.ReadString('\n')
    // 清除因为回车产生的多余空格和换行符
    return strings.TrimSpace(input)
}

// generateID 生成任务的唯一 ID，这里简单使用当前的时间戳
func generateID() string {

```

```

    return strconv.FormatInt(time.Now().Unix(), 10)
}

// addTask 新增任务的核心流程
func addTask() {
    name := readInput("请输入任务名称:")
    desc := readInput("请输入任务描述:")
    cate := readInput("请输入分类(例如: 学习/生活/锻炼):")
    priStr := readInput("请输入优先级(1-5 整数):")

    // strconv.Atoi 尝试把字符串转换为整数 (ASCII to Int)
    priority, _ := strconv.Atoi(priStr)

    // 实例化结构体对象
    task := FitnessTask{
        ID:      generateID(),
        Name:     name,
        Description: desc,
        Category: cate,
        Status:   "待处理",
        Priority: priority,
        CreatedAt: time.Now(),
        Deadline: time.Now().Add(24 * time.Hour), // 默认演示: 设定为 24 小时后截
    }
    止

    // append 是 Go 特有的内建函数, 将新元素追加到切片末尾
    tasks = append(tasks, task)
}

```

```

// Sprintf: 不直接打印到屏幕，而是把变量拼成字符串，交给日志系统
recordLog(fmt.Sprintf("新增任务: [%s] %s", task.ID, task.Name))

fmt.Println("任务添加成功，并已完成数据持久化！")
}

// queryTasks 查看所有任务
func queryTasks() {
    if len(tasks) == 0 {
        fmt.Println("当前没有任何任务。")
        return
    }
    fmt.Println("\n--- 任务列表 ---")
    // 格式化输出表头： %-13s 表示对齐并在右侧填充至 13 个字符
    fmt.Printf("%-13s | %-10s | %-6s | %-6s | %-4s | %-10s\n", "ID", "名称", "分类", "
状态", "优先级", "创建日期")
    fmt.Println(strings.Repeat("-", 65))

    // for range 是 Go 语言遍历切片(数组)最常见的语法
    // _ 是索引用不着，所以用下划线忽略；t 表示当前取出来的单条任务数据
    for _, t := range tasks {
        // t.CreatedAt.Format 是 Go 语言特殊的时间格式化语法 (用固定的 2006-01-02
        15:04:05 表示格式模板)
        fmt.Printf("%-13s | %-10s | %-6s | %-6s | %-4d | %-10s\n", t.ID, t.Name,
        t.Category, t.Status, t.Priority, t.CreatedAt.Format("01-02 15:04"))
    }
}

```

```

// updateTask 更新任务的状态字段
func updateTask() {
    id := readInput("请输入要更新状态的任务 ID: ")

    // i 是当前元素的索引(0, 1, 2...)
    for i, t := range tasks {
        if t.ID == id {
            newStatus := readInput(fmt.Sprintf("输入新状态 (当前: %s, 建议填: 待处理/进行中/已完成): ", t.Status))
            if newStatus != "" {
                // 修改指定索引所在元素的值
                tasks[i].Status = newStatus
                recordLog(fmt.Sprintf("更新任务 [%s] 的状态为: %s", t.ID, newStatus))
                fmt.Println("任务状态更新成功！")
            }
            return
        }
    }
    fmt.Println("错误：未找到指定的任务 ID。")
}

```

```

// deleteTask 从切片中移除任务数据
func deleteTask() {
    id := readInput("请输入要删除的任务 ID: ")
    for i, t := range tasks {
        if t.ID == id {

```

// 在 Go 中删除切片的某个元素，常用的技巧：取你要删除项的前半段 + 后半段，然后拼接起来

// tasks[:i] 获取 [0 到 i-1] 的元素

// tasks[i+1:] 获取 [i+1 到 末尾] 的元素

// ... 表示将其打散依次接进去

tasks = append(tasks[:i], tasks[i+1:]...)

recordLog(fmt.Sprintf("删除任务 [%s]", id))

fmt.Println("任务删除成功！")

return

}

}

fmt.Println("错误：未找到指定的任务 ID。")

}

// main 主函数，整个程序的唯一入口

func main() {

// 第一步：先将硬盘的文件数据反序列化读取到内存里

loadData()

// for { ... } 是 Go 语言里的死循环（等同于 while (true)），用于提供常驻菜单

for {

fmt.Println("\n===== 任务管理系统 =====")

fmt.Println("1. 新增任务")

fmt.Println("2. 查看任务")

fmt.Println("3. 更新状态")

fmt.Println("4. 删除任务")

fmt.Println("5. 校验底层操作记录防篡改链")


```
fmt.Println("0. 退出系统")

fmt.Println("=====")

choice := readInput("请选择操作编号:")

// switch 分支语句，根据用户输入的字符串数字执行对应的函数
switch choice {
case "1":
    addTask()
case "2":
    queryTasks()
case "3":
    updateTask()
case "4":
    deleteTask()
case "5":
    if verifyChain() {
        fmt.Println("验证结果：日志链完整，所有历史数据均未被外部篡改！")
    } else {
        fmt.Println("警告检测：发现链条断裂或指纹不匹配，记录文件可能已被文本编辑器强行修改过！")
    }
case "0":
    fmt.Println("感谢使用，系统已退出。")
    return // 终止整个 main 函数，程序结束
default:
    fmt.Println("无效的输入编号。")
}
```

}

}